

Fondements informatiques I

Cours 5: Types mutables et immutables

Fonctions (suite et fin)

Sorina Ionica `sorina.ionica@uvsq.fr`

Bilal FAYE `bilal.faye@uvsq.fr`

Types mutables et immutables

Deux catégories d'objets

En Python, certains objets peuvent être **modifiés**, d'autres ne peuvent pas l'être.

Objets immutables

Leur contenu ne peut pas être modifié.

- entiers et flottants
- chaînes de caractères
- tuples

Exemple

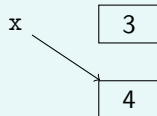
```
x = 3
```

```
x += 1
```



Après l'opération :

```
x = 4
```



Le 3 n'a pas été modifié. Python crée un nouvel entier 4 et x référence maintenant cet objet.

Types mutables et immutables

Les objets mutables

Un objet **mutable** peut être **modifié après sa création**.

Exemple avec une liste

```
liste = [1, 2, 3]  
liste[0] = 5
```

Avant

liste → [1,2,3]

Après modification

liste → [5,2,3]

À retenir

Mutable : on peut modifier l'objet lui-même.

Exemples : listes, ensembles et dictionnaires (qu'on verra plus tard).

Question

Que se passe-t-il lors de l'exécution de chacune de ces instructions ?

Exemple 1

```
mon_tuple = (1, 2, 3)
mon_tuple[0] = 5
```

Exemple 3

```
x = 5
x[0] = 6
```

Exemple 2

```
chaine = "hello"
chaine[0] = "H"
```

Exemple 4

```
liste = [1, 2, 3]
liste[0] = 5
```

Exercice : mettre les mots en majuscule

Écrire une fonction `capitalise(chaine)` qui renvoie une chaîne où la première lettre de chaque mot est en majuscule.

Exercice : mettre les mots en majuscule

Écrire une fonction `capitalise(chaine)` qui renvoie une chaîne où la première lettre de chaque mot est en majuscule.

Exemple

```
def capitalise(chaine):  
    resultat = ""  
    debut_mot = True  
    for c in chaine:  
        if debut_mot and c != " "  
            resultat += c.upper()  
            debut_mot = False  
        else:  
            resultat += c  
        if c == " "  
            debut_mot = True  
    return resultat  
print(capitalise("bonjour tout le monde"))
```

Les variables locales

Définition

Les variables définies dans une fonction sont appelées **variables locales**. Leur **portée** est limitée à la fonction où elles ont été définies : on ne peut pas y faire référence en dehors.

Exemple

```
def incremente(x):  
    x += 1  
    z = 2  
incremente(2)  
print(x)  # Erreur  
print(z)  # Erreur
```

À retenir

x et z n'existent que dans la fonction. Les afficher à l'extérieur provoque une `NameError`.

Évaluation de la fonction

- Les arguments sont des variables locales à la fonction.
- Les arguments sont passés par la copie d'une référence sur l'objet donné en argument.

```
def incremente(x):  
    x += 1  
y = 0  
incremente(y)  
print(y)
```


Évaluation de la fonction

- Les arguments sont des variables locales à la fonction.
- Les arguments sont passés par la copie d'une référence sur l'objet donné en argument.

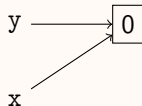
```
def incremente(x):  
    x += 1  
  
y = 0  
incremente(y)  
print(y)
```



Évaluation de la fonction

- Les arguments sont des variables locales à la fonction.
- Les arguments sont passés par la copie d'une référence sur l'objet donné en argument.

```
def incremente(x):  
    x += 1  
y = 0  
incremente(y)  
print(y)
```



Évaluation de la fonction

- Les arguments sont des variables locales à la fonction.
- Les arguments sont passés par la copie d'une référence sur l'objet donné en argument.

```
def incremente(x):  
    x += 1  
y = 0  
incremente(y)  
print(y)
```

y → 0

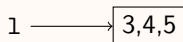
x → 1

Référence sur une liste

À retenir :

- Sur une liste, on peut modifier son contenu depuis la fonction, car objet mutable.

```
def incrementeListe(k):  
    for i in range(len(k)):  
        k[i] += 1  
  
l = [3,4,5]  
incrementeListe(l)  
print(l)
```



Référence sur une liste

À retenir :

- Sur une liste, on peut modifier son contenu depuis la fonction, car objet mutable.

```
def incrementeListe(k):  
    for i in range(len(k)):  
        k[i] += 1  
  
l = [3,4,5]  
incrementeListe(l)  
print(l)
```

l → [3,4,5]

Après appel incrementeListe

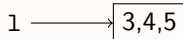
l → [4,5,6]
k → [4,5,6]

Référence sur une liste

À retenir :

- Cependant, on ne peut pas modifier la liste elle même!

```
def supprimeListe(k):  
    k= []  
l = [3,4,5]  
supprimeListe(l)  
print(l)
```

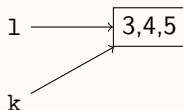


Référence sur une liste

À retenir :

- Cependant, on ne peut pas modifier la liste elle même!

```
def supprimeListe(k):  
    k= []  
l = [3,4,5]  
supprimeListe(l)  
print(l)
```

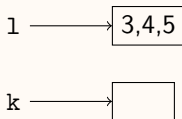


Référence sur une liste

À retenir :

- Cependant, on ne peut pas modifier la liste elle même!

```
def supprimeListe(k):  
    k= []  
l = [3,4,5]  
supprimeListe(l)  
print(l)
```



Utilisation de variables globales

Variable globale

Une variable définie **hors d'une fonction** est une variable **globale**.

- Elle peut être **lue** depuis une fonction.
- En revanche, une affectation dans la fonction crée par défaut une variable **locale**.

Exemple

```
x = 1 # variable globale
def affiche():
    print(x) # utilise la variable globale
affiche() # affiche 1
```

Modifier une variable globale

Le mot-clé global

Pour modifier une variable globale depuis une fonction, il faut utiliser le mot-clé `global`.

Exemple

```
x = 1 # variable globale
def ajoute():
    global x
    x += 1
ajoute()
print(x)          # 2
```

À retenir

L'utilisation de variables globales est généralement **déconseillée** : elle rend le programme plus difficile à comprendre et à maintenir.

Variable globale ou locale ?

Question : qu'affiche le programme ?

```
x = 3
def ajoute():
    x = 5
    x += 1
ajoute()
print(x)
```

Variable globale ou locale ?

Question : qu'affiche le programme ?

```
x = 3
def ajoute():
    x = 5
    x += 1
ajoute()
print(x)
```

Réponse

3

- Le x créé dans ajoute() est **local**.
- Il ne modifie pas le x global.

Arguments avec des valeurs par défaut

Valeur par défaut

On peut donner une valeur par défaut à certains paramètres. Elle est utilisée si l'argument n'est pas fourni lors de l'appel.

Exemple

```
def decrire_personne(pays, age=18, nom="inconnu"):
    print(nom, "a", age, "ans et vit en", pays)
decrire_personne("France")
decrire_personne("Allemagne", 25, "Kurt")
decrire_personne("Italie", 77)
```

- pays est obligatoire.
- age et nom peuvent être omis.

Arguments nommés

Donner le nom des paramètres

On peut préciser le nom des paramètres lors de l'appel. L'ordre des arguments n'est alors plus nécessairement celui de la signature.

Exemple

```
def decrire_personne(pays, age=18, nom="inconnu"):
    print(nom, "a", age, "ans et vit en", pays)
decrire_personne(nom="Kader", age=18, pays="Algérie")
decrire_personne("France", nom="Sylvie")
```

- Les arguments nommés rendent l'appel plus explicite.
- Les paramètres sans valeur par défaut restent obligatoires.

Appel d'une fonction depuis une autre fonction

Principe

Une fonction peut appeler une autre fonction et utiliser la valeur qu'elle retourne ou ce qu'elle affiche.

Exemple

```
def doubler(nombre):  
    return 2 * nombre  
def additionner(a, b):  
    return a + doubler(b)  
print(additionner(2, 3))      # 8
```

- `additionner()` appelle `doubler()`.
- `doubler(3)` retourne 6.
- La fonction `additionner()` calcule alors $2 + 6$.

Appeler une fonction dans une boucle

Question : que va afficher ce programme ?

```
def doubler(nombre):  
    return 2 * nombre  
def doubler_liste(liste):  
    for i in range(len(liste)):  
        doubler(liste[i])  
    return liste  
nombres = [2, 3, 4]  
print(doubler_liste(nombres))
```


Corriger le programme

Question : que va afficher ce programme ?

Correction

```
def doubler(nombre):  
    return 2 * nombre  
def doubler_liste(liste):  
    for i in range(len(liste)):  
        liste[i] = doubler(liste[i])  
    return liste  
nombres = [2, 3, 4]  
print(doubler_liste(nombres))
```

Expressions lambda

Qu'est-ce qu'une expression lambda ?

Une expression lambda permet de créer une **petite fonction** en une seule ligne.

Exemple

```
double = lambda x: 2 * x  
print(double(5))      # 10
```

- x est le paramètre.
- 2 * x est la valeur retournée.

À retenir

Une expression lambda est adaptée aux fonctions **courtes** données par une seule instruction.

Fonction classique et fonction lambda

Une même fonction peut être écrite de deux façons.

Fonction classique

```
def carre(x):  
    return x * x
```

Avec lambda

```
carre = lambda x: x * x
```

Même utilisation

```
print(carre(4))          # 16
```

- La version lambda est plus courte.
- Elle est surtout intéressante pour une fonction très simple.

Quand utiliser une expression lambda ?

Particulièrement utile lorsqu'une petite fonction est utilisée **une seule fois**.

Exemple

```
nombres = [1, 2, 3, 4]
double = lambda x: 2 * x
for nombre in nombres:
    print(double(nombre))
```

On peut aussi utiliser directement l'expression :

```
for nombre in nombres:
    print((lambda x: 2 * x)(nombre))
```

Si une fonction devient longue ou complexe, il vaut mieux utiliser `def` pour la rendre plus lisible.

Exercice de synthèse

Objectif : réutiliser les notions du chapitre.

On considère le programme suivant :

Données

```
nombre = [2, 5, 8, 3, 10]
```

- 1 Écrire une fonction `ajouter_un` qui ajoute 1 à chaque élément de la liste.
- 2 Vérifier que la liste a bien été modifiée.
- 3 Écrire une fonction `creer_double` qui retourne une **nouvelle liste** contenant le double de chaque valeur.
- 4 Écrire une expression `lambda` qui retourne le carré d'un nombre.
- 5 Créer une variable globale `facteur = 2` et écrire une fonction qui utilise cette variable pour multiplier les éléments de la liste.
- 6 Expliquer : quelles modifications concernent la liste **mutable** et quelles valeurs restent **immuables** ?